

Relazione di Calcolo Numerico

Esercizio n.4

Christian Barbarossa

2 Luglio 2026

1 Introduzione Teorica

L'obiettivo del presente esercizio è l'approssimazione numerica della radice positiva dell'equazione non lineare $f(x) = 0$, dove $f(x) = \cos(x) - x$, mediante due procedimenti iterativi: il **metodo di Newton** (o delle tangenti) e il **metodo delle secanti**.

Per risolvere l'equazione $\cos(x) = x$ che non ammette soluzione in forma chiusa; si ricorre a un **procedimento iterativo**, ossia a partire da uno o più punti iniziali si costruisce una successione $\{x_n\}$ che, sotto opportune ipotesi, converge alla radice $\bar{x}$. La rapidità di tale convergenza è quantificata dall'**ordine di convergenza** p : detto $e_n = x_n - \bar{x}$ l'errore al passo n -esimo, il metodo ha ordine $p \geq 1$ se esiste una costante $c > 0$ tale che

$$|e_{n+1}| \leq c |e_n|^p. \quad (1)$$

Per $p = 1$ (convergenza lineare) deve essere $c < 1$; per $p = 2$ la convergenza è detta quadratica. Tanto maggiore è p , tanto più velocemente l'errore si riduce passo dopo passo.

1.1 Localizzazione della radice

La ricerca di uno zero di $f(x) = \cos(x) - x$ equivale a determinare l'ascissa del punto di intersezione tra le curve $y = \cos(x)$ e $y = x$. Poiché $\cos(x) \in [-1, 1]$ per ogni x , tale intersezione può cadere solo nella striscia $x \in [0, 1]$. Valutando f agli estremi di questo intervallo si ottiene $f(0) = 1 > 0$ e $f(1) = \cos(1) - 1 \approx -0.46 < 0$: essendo f continua, per il **teorema degli zeri** (o di Bolzano) esiste almeno una radice $\bar{x} \in (0, 1)$. Inoltre $f'(x) = -\sin(x) - 1 \leq 0$ su tutto $\mathbb{R}$, e strettamente negativa in $(0, 1)$: la funzione è dunque monotona decrescente e la radice è **unica**.

1.2 Il Metodo di Newton

Supponendo f derivabile con derivata non nulla, si approssima la funzione nell'intorno del punto corrente x_n con il suo polinomio di Taylor di primo grado:

$$p(x) = f(x_n) + f'(x_n)(x - x_n). \quad (2)$$

Assumendo come nuova iterata lo zero di tale retta tangente si ricava il procedimento:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad (3)$$

con funzione iteratrice $g(x) = x - f(x)/f'(x)$. Geometricamente, x_{n+1} è l'intersezione con l'asse delle ascisse della retta tangente al grafico di f nel punto $(x_n, f(x_n))$: per questo il metodo è anche detto **metodo delle tangenti**.

Derivando la funzione iteratrice si ottiene

$$g'(x) = \frac{f(x)f''(x)}{[f'(x)]^2}, \quad (4)$$

che si annulla nella radice $\bar{x}$ (dove $f(\bar{x}) = 0$) a patto che $f'(\bar{x}) \neq 0$. Essendo $g'(\bar{x}) = 0$, la convergenza è **almeno quadratica**: se f'' è continua e $f'(\bar{x}) \neq 0$ il metodo ha ordine $p = 2$, con

$$|e_{n+1}| \approx \frac{1}{2} \left| \frac{f''(\bar{x})}{f'(\bar{x})} \right| |e_n|^2. \quad (5)$$

Il prezzo di tale rapidità è che **ogni iterazione richiede due valutazioni**: quella di $f(x_n)$ e quella di $f'(x_n)$. Nel caso in esame le derivate sono note analiticamente, $f'(x) = -\sin(x) - 1$ e $f''(x) = -\cos(x)$; poiché nell'intorno della radice $f'(\bar{x}) \approx -1.67 \neq 0$, sono soddisfatte le ipotesi che garantiscono la convergenza quadratica.

1.3 Il Metodo delle secanti

Quando la derivata non è disponibile, oppure è troppo onerosa da calcolare, la si può approssimare con il rapporto incrementale costruito sulle due iterate più recenti:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}. \quad (6)$$

Sostituendo tale approssimazione nella formula di Newton si ottiene il **metodo delle secanti**:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}. \quad (7)$$

Geometricamente x_{n+1} è lo zero della retta **secante** il grafico di f nei due punti $(x_{n-1}, f(x_{n-1}))$ e $(x_n, f(x_n))$; per innescare il procedimento sono perciò necessari **due punti iniziali** x_0 e x_1 .

Si dimostra che, in un intorno della radice, l'ordine di convergenza è pari al rapporto aureo

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618, \quad (8)$$

per cui la convergenza è **superlineare**, leggermente più lenta di quella di Newton. Il vantaggio compensativo è che **ogni iterazione costa una sola valutazione** di f (quella nel nuovo punto), poiché $f(x_{n-1})$ è già stata calcolata al passo precedente e viene riutilizzata.

1.4 Criterio di arresto

Non potendo usare l'errore vero $|\bar{x} - x_n|$ (la radice $\bar{x}$ è incognita), lo si **stima** con la differenza fra due iterate successive $|x_{n+1} - x_n|$, quantità che approssima bene l'errore per i metodi a convergenza superlineare. Per controllare in modo uniforme sia la componente assoluta sia quella relativa dell'errore, e quindi gestire correttamente radici di grandezza qualsiasi, si adotta il criterio sull'**errore misto**:

$$|x_{n+1} - x_n| < atol + rtol |x_{n+1}|, \quad (9)$$

equivalente alla forma

$$\frac{|x_{n+1} - x_n|}{1 + \frac{rtol}{atol} |x_{n+1}|} < atol. \quad (10)$$

Con la scelta $atol = rtol = 10^{-8}$ richiesta dalla traccia, la (9) coincide inoltre con il criterio impiegato internamente dalla routine `scipy.optimize.newton`: ciò rende immediato e coerente il confronto richiesto al Punto 6.

2 Codice Sorgente Python

Di seguito viene riportato lo script Python sviluppato per l'implementazione dei due metodi, la localizzazione grafica della radice, la visualizzazione dei passi e i confronti numerici richiesti. Le tabelle prodotte a tempo di esecuzione sono formattate mediante la libreria `tabulate`.

```
1
2 import numpy as np                                # array e funzioni matematiche vettoriali
3 import matplotlib.pyplot as plt                  # grafici (stessa convenzione delle lezioni)
4 from scipy.optimize import newton as scipy_newton # routine di riferimento (punto 6)
5 from tabulate import tabulate                   # stampa ordinata delle tabelle
6
7
8 # intestazione comune alle tabelle di iterazione dei due metodi
9 _HEADER_IT = ['num_iterazioni', 'x_n', 'f(x_n)', '|x_n - x_(n-1)|']
10 # formato per colonna: nit intero, x_n con 12 decimali, residuo ed errore in
11 # notazione scientifica
12 _FMT_IT = ('d', '.12f', '.6e', '.6e')
13
14
15 # -----
16 # Funzione di cui cercare lo zero e sua derivata prima
17 # -----
18 def f(x):
19     # funzione: f(x) = cos(x) - x
20     return np.cos(x) - x
21
22
23 def df(x):
24     # Derivata prima: f'(x) = -sin(x) - 1
25     return -np.sin(x) - 1.0
26
27
28 # -----
29 # Metodo di Newton (o delle tangenti)
30 # -----
31 def newton(f, df, x0, atol=1e-8, rtol=1e-8, nmax=100, displayit=1):
32     # --- inizializzazione ---
33     x = x0                                # iterata corrente: si parte da x0
34     fx = f(x)                             # f(x0): la si calcola una volta sola
35     nfun = 1                             # contatore valutazioni di funzione (f(x0) -> 1)
36     nit = 0                              # contatore iterazioni
37     exitflag = 0                          # 0 finche' non si raggiunge la convergenza
38     stima_err = np.inf                    # stima errore iniziale "grande" per entrare nel ciclo
39     storia = []                          # raccoglie le righe della tabella di iterazione
40
41     # --- ciclo iterativo ---
42     while nit < nmax:
43         dfx = df(x)                       # salvaguardia sul numero massimo di passi
44         nfun += 1                         # valuto la derivata nel punto corrente
45                                         # -> una valutazione di funzione in piu'
46
47         if dfx == 0:                      # derivata nulla: la tangente e' orizzontale,
48             print('Derivata nulla: impossibile proseguire.')
49             break                          # il passo non e' definito -> esco
50
51         dx = fx / dfx                     # passo di Newton: f(x_n) / f'(x_n)
52         x = x - dx                       # nuova iterata: x_(n+1) = x_n - f/f'
53         fx = f(x)                       # f nella nuova iterata (serve al passo
54         nfun += 1                       # successivo e per riportare f(x_n) finale)
55         nit += 1                         # iterazione completata
56         stima_err = abs(dx)               # stima errore = |x_(n+1) - x_n| = |dx|
57
58         storia.append([nit, x, fx, stima_err]) # memorizzo la riga per la tabella
59
60     # criterio d'arresto sull'errore misto:
61     # |x_(n+1) - x_n| < atol + rtol * |x_(n+1)|
62     if stima_err < atol + rtol * abs(x):
63         exitflag = 1                     # convergenza raggiunta -> esco dal ciclo
64         break
```

```

65     if displayit:                                     # stampo la tabella delle iterazioni
66         print(tabulate(storia, headers=_HEADER_IT,
67                        floatfmt=_FMT_IT, tablefmt='fancy_grid'))
68
69     return (x, fx, exitflag, nfun, nit, stima_err)
70
71
72 # -----
73 # Metodo delle secanti
74 # -----
75 def secanti(f, x0, x1, atol=1e-8, rtol=1e-8, nmax=100, displayit=1):
76     # --- inizializzazione: servono due punti e i rispettivi valori di f ---
77     xprec = x0                                         # x(n-1): penultima iterata
78     fprec = f(xprec)                                   # f(x(n-1))
79     x = x1                                             # xn: ultima iterata
80     fx = f(x)                                          # f(xn)
81     nfun = 2                                           # gia' eseguite 2 valutazioni: f(x0) e f(x1)
82     nit = 0
83     exitflag = 0
84     stima_err = np.inf
85     storia = []                                       # raccoglie le righe della tabella di iterazione
86
87     # --- ciclo iterativo ---
88     while nit < nmax:
89         den = fx - fprec                               # denominatore del rapporto incrementale
90         if den == 0:                                   # i due valori di f coincidono: secante
91             print('Denominatore nullo: impossibile proseguire.') # orizzontale
92             break
93
94         dx = fx * (x - xprec) / den                    # passo delle secanti
95
96         # aggiorno la "memoria" PRIMA di sovrascrivere x:
97         xprec = x                                     # la x corrente diventa la penultima
98         fprec = fx                                    # e con essa il suo valore di f
99         x = x - dx                                    # nuova iterata x(n+1)
100        fx = f(x)                                       # f nella nuova iterata: UNICA valutazione
101        nfun += 1                                       # del passo -> una sola valutazione di f
102        nit += 1
103        stima_err = abs(dx)                            # stima errore = |x(n+1) - xn|
104
105        storia.append([nit, x, fx, stima_err])          # memorizzo la riga per la tabella
106
107        # stesso criterio d'arresto sull'errore misto usato per Newton
108        if stima_err < atol + rtol * abs(x):
109            exitflag = 1
110            break
111
112        if displayit:                                   # stampo la tabella delle iterazioni
113            print(tabulate(storia, headers=_HEADER_IT,
114                           floatfmt=_FMT_IT, tablefmt='fancy_grid'))
115
116    return (x, fx, exitflag, nfun, nit, stima_err)
117
118
119 # -----
120 # Punto 1 - localizzazione grafica della radice positiva
121 # -----
122 def localizza(a=0.0, b=1.5, x0=0.5):
123     """
124     Disegna f(x)=cos(x)-x sull'intervallo [a,b] per localizzare la radice
125     positiva, evidenzia il punto iniziale x0 e SEGNA lo zero della funzione
126     (individuato dal cambio di segno tra i punti campionati). Stampa inoltre
127     il controllo del cambio di segno agli estremi di [0,1].
128     """
129     xp = np.linspace(a, b, 200)                       # griglia di punti per il grafico
130     yp = f(xp)                                         # valori di f sulla griglia (servono per lo zero)
131
132     plt.figure(figsize=(7, 4))
133     plt.plot(xp, yp, label=r'$f(x)=\cos(x)-x$')         # grafico di f
134     plt.plot(xp, 0 * xp, 'k--', linewidth=0.8)        # asse x (retta y = 0)
135     plt.axvline(x0, color='red', linestyle=':',       # punto iniziale
136                 label=r'$x_0=%.2f$' % x0)
137

```

```

138 # --- individuo e segno lo zero della funzione ---
139 # np.sign(yp) da' il segno (+1/-1) in ogni punto; np.diff e' diverso da zero
140 # solo dove il segno cambia: tra quei due punti la funzione attraversa lo zero.
141 idx = np.where(np.diff(np.sign(yp)) != 0)[0]
142 for i in idx:
143     # stima dell'ascissa dello zero per interpolazione lineare fra i due punti
144     xz = xp[i] - yp[i] * (xp[i + 1] - xp[i]) / (yp[i + 1] - yp[i])
145     plt.plot(xz, 0, 'go', markersize=10, label=r'radice $\approx %.4f$' % xz)
146
147 plt.xlabel('x')
148 plt.ylabel('f(x)')
149 plt.title('Localizzazione grafica della radice positiva')
150 plt.legend()
151 plt.grid(True)
152 plt.show()
153
154 # Controllo numerico del cambio di segno agli estremi di [0, 1]:
155 # cos(x) in [-1,1] interseca y=x solo per x in [0,1]; f(0)=1>0, f(1)<0,
156 # quindi per il teorema degli zeri la radice cade in (0,1).
157 print('f(0)      = %.4f' % f(0.0))
158 print('f(1)      = %.4f' % f(1.0))
159 print('f(0)*f(1) = %.4f -> segni opposti: esiste una radice in (0,1)'
160       % (f(0.0) * f(1.0)))
161
162
163 # -----
164 # Visualizzazione grafica dei passi dei due metodi
165 # -----
166 # colori distinti per i passi successivi (ben staccati dal nero della curva)
167 _COLORI_PASSI = ['#d62728', '#ff7f0e', '#9467bd', '#8c564b', '#e377c2', '#1f77b4']
168
169
170 def _griglia_plot(xs, extra=0.06):
171     """Intervallo dell'asse x su cui disegnare f, ricavato dalle iterate xs."""
172     lo, hi = min(xs), max(xs)
173     m = 0.18 * (hi - lo) + extra # un po' di margine ai lati
174     return np.linspace(lo - m, hi + m, 400)
175
176 def _etichetta_iterate(xs):
177     """Scrive x_0, x_1, ... sotto l'asse x, saltando le iterate troppo vicine
178     fra loro (che, per la rapida convergenza, si sovrapporrebbero)."""
179     soglia = 0.03 * (max(xs) - min(xs))
180     fatte = []
181     for k, xk in enumerate(xs):
182         if all(abs(xk - lx) > soglia for lx in fatte):
183             plt.annotate(r'$x_{%d}$' % k, (xk, 0), textcoords='offset points',
184                        xytext=(0, -17), ha='center', fontsize=12)
185             fatte.append(xk)
186
187 def plot_newton(f, df, x0, atol=1e-8, rtol=1e-8, nmax=100, n_passi=None):
188     """
189     Visualizza i passi del metodo di Newton. Per ogni iterazione disegna la
190     retta TANGENTE nel punto corrente (x_n, f(x_n)) e la sua intersezione con
191     l'asse x, che fornisce l'iterata successiva x_(n+1). Con n_passi si puo'
192     limitare il numero di passi disegnati.
193     """
194     # rieseguo l'iterazione raccogliendo tutte le iterate x0, x1, x2, ...
195     xs, x = [x0], x0
196     for _ in range(nmax):
197         d = df(x)
198         if d == 0:
199             break
200         xnew = x - f(x) / d
201         xs.append(xnew)
202         if abs(xnew - x) < atol + rtol * abs(xnew):
203             break
204         x = xnew
205     if n_passi is not None:
206         xs = xs[:n_passi + 1]
207
208     xp = _griglia_plot(xs)
209     plt.figure(figsize=(8, 5))
210     plt.plot(xp, f(xp), 'k-', linewidth=2, label=r'$f(x)=\cos(x)-x$') # curva (nera)

```

```

211 plt.axhline(0, color='gray', linewidth=0.8) # asse x
212
213 for k in range(len(xs) - 1):
214     c = _COLORI_PASSI[k % len(_COLORI_PASSI)]
215     xn, xn1 = xs[k], xs[k + 1]
216     plt.plot([xn, xn1], [f(xn), 0], '--', color=c, lw=1.8) # tangente -> asse x
217     plt.plot([xn1, xn1], [0, f(xn1)], ':', color=c, lw=1.2) # verticale al nuovo
punto
218     plt.plot(xn, f(xn), 'o', color=c, markersize=8) # punto sulla curva
219
220 _etichetta_iterate(xs)
221 plt.plot(xs[-1], 0, '*', color='limegreen', markersize=18, # radice
222          markeredgecolor='k', markeredgewidth=0.5, label='radice')
223 plt.xlabel('x'); plt.ylabel('f(x)')
224 plt.title('Passi del metodo di Newton (rette tangenti)')
225 plt.legend(); plt.grid(True, alpha=0.3)
226 plt.show()
227
228 def plot_secanti(f, x0, x1, atol=1e-8, rtol=1e-8, nmax=100, n_passi=None):
229     """
230     Visualizza i passi del metodo delle secanti. Per ogni iterazione disegna la
231     retta SECANTE passante per gli ultimi due punti e la sua intersezione con
232     l'asse x, che fornisce l'iterata successiva.
233     """
234     # rieseguo l'iterazione raccogliendo le iterate x0, x1, x2, ...
235     xs = [x0, x1]
236     xprec, fprec, x, fx = x0, f(x0), x1, f(x1)
237     for _ in range(nmax):
238         den = fx - fprec
239         if den == 0:
240             break
241         xnew = x - fx * (x - xprec) / den
242         xs.append(xnew)
243         if abs(xnew - x) < atol + rtol * abs(xnew):
244             break
245         xprec, fprec, x, fx = x, fx, xnew, f(xnew)
246     if n_passi is not None:
247         xs = xs[:n_passi + 2]
248
249     xp = _griglia_plot(xs)
250     plt.figure(figsize=(8, 5))
251     plt.plot(xp, f(xp), 'k-', linewidth=2, label=r'$f(x)=\cos(x)-x$') # curva (nera)
252     plt.axhline(0, color='gray', linewidth=0.8) # asse x
253
254     for k in range(1, len(xs) - 1):
255         c = _COLORI_PASSI[(k - 1) % len(_COLORI_PASSI)]
256         xa, xb, xc = xs[k - 1], xs[k], xs[k + 1] # due punti + intercetta
257         xx = np.array([min(xa, xb, xc), max(xa, xb, xc)])
258         m = (f(xb) - f(xa)) / (xb - xa) # pendenza della secante
259         plt.plot(xx, f(xa) + m * (xx - xa), '--', color=c, lw=1.8) # secante
260         plt.plot([xc, xc], [0, f(xc)], ':', color=c, lw=1.2) # verticale
261         plt.plot([xa, xb], [f(xa), f(xb)], 'o', color=c, markersize=8) # i due punti
262
263     _etichetta_iterate(xs)
264     plt.plot(xs[-1], 0, '*', color='limegreen', markersize=18, # radice
265            markeredgecolor='k', markeredgewidth=0.5, label='radice')
266     plt.xlabel('x'); plt.ylabel('f(x)')
267     plt.title('Passi del metodo delle secanti')
268     plt.legend(); plt.grid(True, alpha=0.3)
269     plt.show()
270
271 # -----
272 # Esecuzione: punti 3-4-5-6
273 # -----
274 if __name__ == '__main__':
275
276     # ---- Punti 3 e 4: dati del problema ----
277     x0 = 0.5 # punto iniziale per Newton e PRIMO punto per le secanti
278     x1 = 1.0 # secondo punto iniziale, richiesto dal metodo delle secanti
279     atol = 1e-8 # tolleranza assoluta
280     rtol = 1e-8 # tolleranza relativa
281
282     # ---- Punto 1: localizzazione grafica ----

```

```

283 print('=== PUNTO 1: localizzazione grafica ===')
284 localizza(0.0, 1.5, x0)
285 print()
286
287 # ---- Esecuzione del metodo di Newton ----
288 print('=== METODO DI NEWTON ===')
289 xN, fxN, flagN, nfunN, nitN, errN = newton(f, df, x0, atol, rtol, displayit=1)
290 print('Radice approssimata : %.15f (convergenza flag = %d)' % (xN, flagN))
291 print()
292
293 # ---- Esecuzione del metodo delle secanti ----
294 print('=== METODO DELLE SECANTI ===')
295 xS, fxS, flagS, nfunS, nitS, errS = secanti(f, x0, x1, atol, rtol, displayit=1)
296 print('Radice approssimata : %.15f (convergenza flag = %d)' % (xS, flagS))
297 print()
298
299 # ---- Visualizzazione grafica dei passi dei due metodi ----
300 print('=== VISUALIZZAZIONE DEI PASSI (chiudere ogni finestra per proseguire) ===')
301 plot_newton(f, df, x0, atol, rtol)
302 plot_secanti(f, x0, x1, atol, rtol)
303 print()
304
305 # ---- Punto 5: confronto fra i due metodi ----
306 # Ogni cella viene pre-formatata come stringa, così ogni indicatore ha il
307 # suo formato (interi, notazione scientifica, decimali). disable_numparse
308 # impedisce a tabulate di reinterpretare e riformattare queste stringhe.
309 print('=== PUNTO 5: confronto Newton / Secanti ===')
310 tab5 = [
311     ['numero di iterazioni', '%d' % nitN, '%d' % nitS],
312     ['valutazioni di funzione (nfun)', '%d' % nfunN, '%d' % nfunS],
313     ['f(x_n) ultima iterata', '%.3e' % fxN, '%.3e' % fxS],
314     ['errore stimato finale', '%.3e' % errN, '%.3e' % errS],
315     ['radice approssimata', '%.12f' % xN, '%.12f' % xS],
316 ]
317 print(tabulate(tab5, headers=['', 'Newton', 'Secanti'], tablefmt='fancy_grid',
318                 colalign=('left', 'right', 'right'), disable_numparse=True))
319 print()
320
321 # ---- Punto 6: confronto con scipy.optimize.newton ----
322 # Variante "tangenti": con fprime=df scipy esegue il metodo di Newton.
323 radN, infoN = scipy_newton(f, x0, fprime=df,
324                             tol=atol, rtol=rtol, full_output=True)
325 # Variante "secanti": senza fprime ma con x1 scipy esegue il metodo delle secanti.
326 radS, infoS = scipy_newton(f, x0, x1=x1,
327                             tol=atol, rtol=rtol, full_output=True)
328
329 print('=== PUNTO 6: confronto con scipy.optimize.newton ===')
330 tab6 = [
331     ['iterazioni',
332      '%d' % nitN, '%d' % infoN.iterations,
333      '%d' % nitS, '%d' % infoS.iterations],
334     ['valutazioni di funzione',
335      '%d' % nfunN, '%d' % infoN.function_calls,
336      '%d' % nfunS, '%d' % infoS.function_calls],
337     ['radice',
338      '%.10f' % xN, '%.10f' % radN,
339      '%.10f' % xS, '%.10f' % radS],
340 ]
341 print(tabulate(tab6,
342                 headers=['', 'Newton', 'sp.Newton', 'Secanti', 'sp.Secanti'],
343                 tablefmt='fancy_grid', disable_numparse=True,
344                 colalign=('left', 'right', 'right', 'right', 'right')))

```

Listing 1: Implementazione dei metodi di Newton e delle secanti, con visualizzazione dei passi e confronto.

3 Analisi dei Risultati Numerici

L'esecuzione dello script conferma quanto previsto dalla teoria. La Figura 1 mostra il risultato del Punto 1: la funzione $f(x) = \cos(x) - x$ attraversa l'asse delle ascisse una sola volta, in corrispondenza della radice positiva, qui individuata automaticamente dal cambio di segno fra i punti campionati e segnata con un marcatore. Si conferma inoltre che il punto iniziale $x_0 = 0.5$ le è sufficientemente vicino. Tutti i procedimenti convergono alla medesima radice $\bar{x} \approx 0.7390851332$.

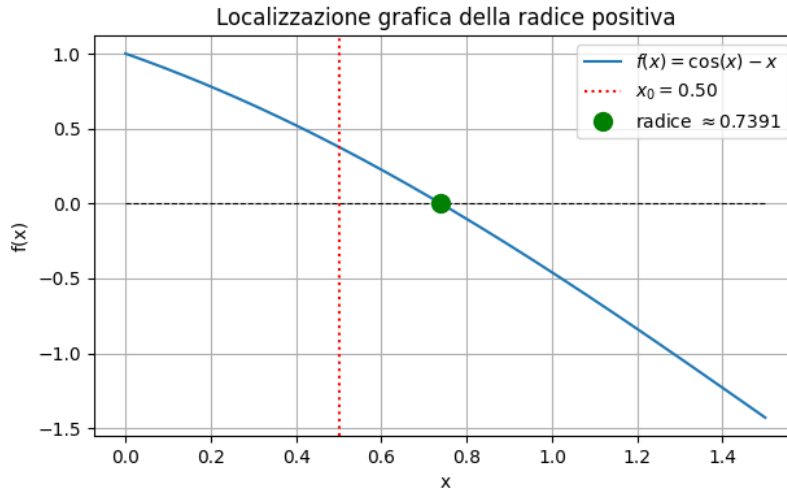


Figura 1: Localizzazione grafica della radice positiva di $f(x) = \cos(x) - x$, con lo zero individuato dal cambio di segno e il punto iniziale x_0 .

3.1 Andamento delle iterazioni

Le Tabelle 1 e 2 riportano, passo per passo, l'iterata x_n , il residuo $f(x_n)$ e la stima dell'errore $|x_n - x_{n-1}|$ per i due metodi.

Tabella 1: Iterazioni del metodo di Newton ($x_0 = 0.5$).

n	x_n	$f(x_n)$	$ x_n - x_{n-1} $
1	0.755222417106	-2.71033×10^{-2}	2.55222×10^{-1}
2	0.739141666150	-9.46154×10^{-5}	1.60808×10^{-2}
3	0.739085133921	-1.18098×10^{-9}	5.65322×10^{-5}
4	0.739085133215	0	7.05646×10^{-10}

Tabella 2: Iterazioni del metodo delle secanti ($x_0 = 0.5, x_1 = 1$).

n	x_n	$f(x_n)$	$ x_n - x_{n-1} $
1	0.725481587064	2.26984×10^{-2}	2.74518×10^{-1}
2	0.738398620137	1.14878×10^{-3}	1.29170×10^{-2}
3	0.739087210821	-3.47711×10^{-6}	6.88591×10^{-4}
4	0.739085132900	5.27269×10^{-10}	2.07792×10^{-6}
5	0.739085133215	3.33067×10^{-16}	3.15048×10^{-10}

3.2 Visualizzazione grafica dei passi

Le Figure 2 e 3 illustrano geometricamente i passi compiuti dai due metodi, rendendo visibile la rapidità di convergenza già osservata nelle tabelle. In entrambe la curva $f(x)$ è disegnata in nero, mentre ogni passo è tracciato con un colore distinto; il marcatore a stella indica la radice.

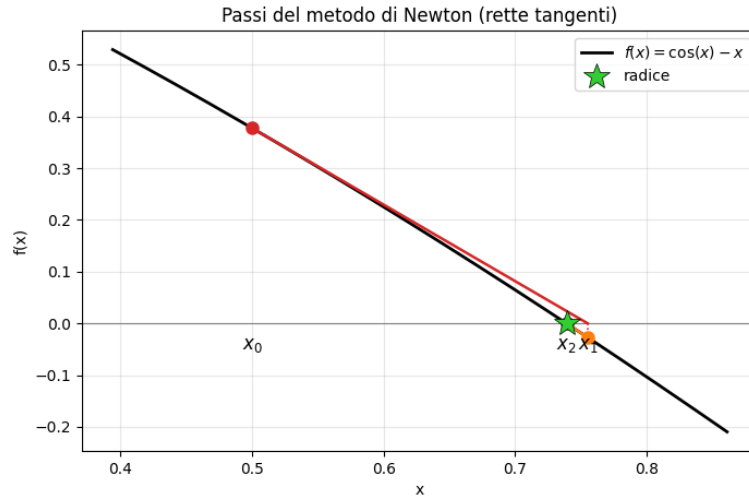


Figura 2: Passi del metodo di Newton: per ogni iterazione la retta tangente in $(x_n, f(x_n))$ interseca l'asse delle ascisse nell'iterata successiva x_{n+1} .

Per il metodo di Newton (Figura 2) si osserva come la tangente costruita nel punto iniziale $x_0 = 0.5$ intersechi l'asse x già in prossimità della radice: la prima iterata x_1 è praticamente sovrapposta allo zero e le successive sono da esso indistinguibili. È la traduzione grafica della convergenza quadratica, ossia la retta tangente costituisce un'eccellente

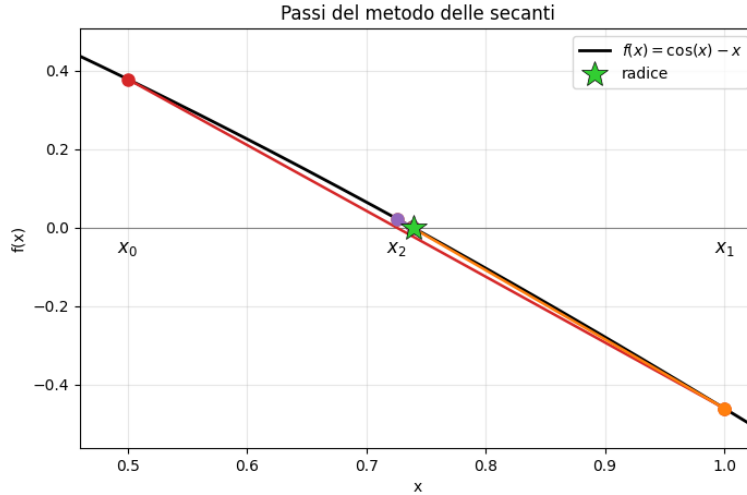


Figura 3: Passi del metodo delle secanti: per ogni iterazione la retta secante per gli ultimi due punti interseca l'asse delle ascisse nell'iterata successiva.

approssimazione locale della funzione, per cui il metodo "salta" quasi immediatamente sulla soluzione. Nel metodo delle secanti (Figura 3) i due punti iniziali $x_0 = 0.5$ e $x_1 = 1$ si trovano da parti opposte rispetto allo zero; la secante che li congiunge taglia l'asse delle ascisse in x_2 , già molto vicino alla radice, e i passi seguenti si addensano rapidamente su di essa. Si noti che, essendo $f(x) = \cos(x) - x$ quasi rettilinea in questo intervallo, le rette tangenti e secanti risultano quasi sovrapposte al grafico della funzione: proprio questa quasi-linearità spiega l'elevata velocità di convergenza di entrambi i metodi.

3.3 Confronto fra i due metodi

La Tabella 3 riassume i quattro indicatori richiesti dal Punto 5.

Tabella 3: Confronto fra il metodo di Newton e il metodo delle secanti.

Indicatore	Newton	Secanti
Numero di iterazioni	4	5
Valutazioni di funzione	9	7
$f(x_n)$ nell'ultima iterata	0	3.33×10^{-16}
Errore stimato finale	7.06×10^{-10}	3.15×10^{-10}
Radice approssimata	0.739085133215	0.739085133215

3.4 Confronto con `scipy.optimize.newton`

La Tabella 4 confronta i risultati delle implementazioni proposte con quelli della routine di riferimento, eseguita con gli stessi parametri ($atol = rtol = 10^{-8}$).

Tabella 4: Confronto con la routine di riferimento `scipy.optimize.newton`.

Indicatore	Newton	sp. Newton	Secanti	sp. Secanti
Iterazioni	4	4	5	5
Valutazioni di funzione	9	8	7	6
Radice	0.7390851332	0.7390851332	0.7390851332	0.7390851332

3.5 Commento ai Dati

Dall'analisi delle tabelle e delle figure precedenti emergono le seguenti considerazioni:

- **Convergenza quadratica di Newton.** Osservando la colonna $|x_n - x_{n-1}|$ della Tabella 1, la stima dell'errore si riduce a ogni passo approssimativamente come il quadrato di quella precedente ($2.55 \times 10^{-1} \rightarrow 1.61 \times 10^{-2} \rightarrow 5.65 \times 10^{-5} \rightarrow 7.06 \times 10^{-10}$). L'ordine di convergenza $p = 2$ è verificato empiricamente: raddoppiando le cifre esatte a ogni iterazione asintotica, l'algoritmo converge alla precisione di macchina in sole 4 iterazioni.
- **Convergenza superlineare delle secanti.** Anche l'errore del metodo delle secanti (Tabella 2) decresce in modo superlineare, ma con ordine $p \approx 1.618$ inferiore a quello di Newton; di conseguenza il metodo necessita di un'iterazione in più (5 contro 4) per soddisfare il medesimo criterio d'arresto.
- **Costo computazionale e compromesso fra i due metodi.** Pur richiedendo meno iterazioni, il metodo di Newton effettua complessivamente *più* valutazioni: le 9 valutazioni si suddividono in 5 valutazioni di f ($= n_{it} + 1$) e 4 di f' ($= n_{it}$). Il metodo delle secanti, non utilizzando mai la derivata, esegue 7 valutazioni, tutte di f . Emerge così il compromesso caratteristico: le secanti sono leggermente più lente in numero di passi, ma globalmente più economiche in numero di valutazioni. La scelta del metodo dipende quindi dal costo relativo di f' rispetto a f : se la derivata è onerosa (o non disponibile in forma chiusa), il metodo delle secanti risulta preferibile.
- **Residuo finale.** In entrambi i casi il valore $f(x_n)$ nell'ultima iterata si attesta a livello della precisione di macchina (esattamente 0 per Newton, 3.33×10^{-16} per le secanti), a conferma del fatto che le iterate finali annullano di fatto la funzione.
- **Coerenza con `scipy`.** Il numero di iterazioni delle implementazioni proposte **coincide esattamente** con quello di `scipy` per entrambi i metodi, poiché viene adottato il medesimo criterio d'arresto sull'errore misto. Il conteggio delle valutazioni di funzione risulta invece superiore di un'unità rispetto a `scipy` (9 contro 8, 7 contro 6): non si tratta di un'inefficienza, bensì di una scelta deliberata, ossia le routine sviluppate valutano f anche nell'ultima iterata accettata, per poter restituire $f(x_n)$ come richiesto al Punto 5, mentre `scipy` verifica la convergenza *prima* di tale ultima valutazione e termina immediatamente.